TCS NQT 2026 Batch ON-Campus All Shifts' Coding Solutions (C++)

Note 1: The questions in this might feel overwhelming, confusing, complicated for no reason. But that's exactly how the coding questions come in TCS NQT.

Note 2: Unlike the hiring assessments of most other companies, in TCS NQT, you have to code from scratch (and not just the function logic code), including input and output statements, and that's how the solutions have been given in this pdf.

12th November Shift 1

Q1: The Ritual of Circular Kriya

Problem Description:
In an ancient spiritual tradition, there are N sacred energy centers arranged in a perfect **circular formation**. These centers are numbered sequentially from 1 to N. Center 1 is adjacent to Center 2, Center 2 is adjacent to Center 3, and due to the circular nature, **Center N is adjacent to Center 1**.

A monk practices a specialized technique called **"Kriya"**. When a monk performs Kriya on a specific energy center X, he absorbs the energy value of that center (which is equal to its number, X).

However, performing Kriya comes with a significant cost:

- The center X itself, its immediate predecessor (X-1), and its immediate successor (X+1) all **vanish** instantly.
- Once a center has vanished, it can no longer be used to perform Kriya, nor can its energy be absorbed.

The monk wants to perform a series of Kriyas on different centers to maximize the total absorbed energy. Furthermore, there is a **Multiplication Factor M** provided for the ritual. The final result of the ritual is the maximum possible energy sum multiplied by M.

Your task is to calculate the final spiritual energy result for the given N and M.

Constraints:

- $3 \le N \le 10^9$ (Note: For $N < 3$, the neighbors overlap significantly).
- $1 \le M \le 100$

Input Format:

- A single line containing two space-separated integers, N and M.

Output Format:

- A single integer representing the maximum energy sum multiplied by M.

Example Test Cases

Example 1

Input:

```
5 4
```

Explanation:
Energy centers: `[1, 2, 3, 4, 5]` in a circle.

- If the monk performs Kriya on center **5**, centers `4`, `5`, and `1` vanish. (Sum = 5). Remaining: `[2, 3]`.
- Now, he can perform Kriya on center **3** (Sum = 5 + 3 = 8). Center `2` vanishes.
- Total Max Sum = 8.
- Final Result = 8 × M = 8 × 4 = 32.

Output:

```
32
```

Example 2

Input:

```
10 4
```

Explanation:
Energy centers: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]` in a circle.
To maximize the sum while ensuring no selected centers are neighbors:

- Perform Kriya on: **10, 8, 6, 4, 2**.
- Total Max Sum = 10 + 8 + 6 + 4 + 2 = 30.
- Final Result = 30 × 4 = 120.

Output:

```
120
```

Concept Mapping

- **Graph Theory (Independent Set):** The circular arrangement of nodes forms a **Cycle Graph (C_n)**. The restriction that vanished neighbors cannot be picked is equivalent to finding the **Maximum Weight Independent Set** on this cycle graph.
- **Greedy Algorithm & Parity:** The problem leverages a greedy choice based on parity (odd/even). Since elements are sequentially numbered, choosing all even numbers guarantees the highest possible sum for even N, and choosing alternating odds downwards from N guarantees the highest sum for odd N.
- **Arithmetic Progressions:** Instead of iterating $O(N)$ times, the solution utilizes the mathematical formulas for the sum of the first K even numbers ($K \times (K+1)$) and the sum of the first K odd numbers (K^2) to achieve an $O(1)$ time complexity.
- **Large Integer Arithmetic:** Because $N \leq 10^9$, the maximum sum $\approx 2.5 \times 10^{17}$, and multiplied by $M \leq 100$, the final answer can reach $\approx 2.5 \times 10^{19}$. This narrowly exceeds the maximum limit of a standard 64-bit signed integer. The code uses arbitrary-precision integers (like `BigInteger` in Java, `__int128` in C++, and default big ints in Python) to prevent silent overflow errors.

Solution Logic

1. **Input Reading:** Capture the total number of energy centers `N` and the multiplier `M`.
2. **Even Number of Centers ($N\%2 == 0$):**
   - In a circle of an even number of elements, you can perfectly select half the items without any neighbors touching.
   - Comparing the set of all evens $\{2, 4, 6, ..., N\}$ to all odds $\{1, 3, 5, ..., N-1\}$, the evens obviously yield a greater sum.
   - The total number of even elements is $K = N/2$.
   - The sum of the first K even numbers is calculated instantly using the formula: `Max Sum = K * (K + 1)`.
3. **Odd Number of Centers ($N\%2! = 0$):**
   - With an odd number of items, taking alternate items will eventually cause the first and last selected items to be adjacent.
   - To maximize the weight, we start taking the highest possible values downwards: $N, N-2, N-4, \ldots, 3$.
   - We are forced to skip 1 because picking N causes $N-1$ and 1 to vanish.
   - The total number of odd elements up to N is $K = (N+1)/2$.
   - The sum of the first K odd numbers is K^2. Since we must exclude the number 1, the formula becomes: `Max Sum = (K * K) - 1`.
4. **Final Output:** The optimal sum derived above is multiplied by M and printed to the console.

C++ Solution

```
#include <iostream>
#include <string>

using namespace std;
```

```
void print128(__int128 n) {
    if (n == 0) {
        cout << 0 << endl;
        return;
    }
    string s;
    while (n > 0) {
        s += (char)('0' + (n % 10));
        n /= 10;
    }
    for (int i = s.length() - 1; i >= 0; i--) {
        cout << s[i];
    }
    cout << endl;
}

int main() {
    long long n_in, m_in;
    if (cin >> n_in >> m_in) {
        __int128 n = n_in;
        __int128 m = m_in;
        __int128 max_sum = 0;

        if (n % 2 == 0) {
            __int128 k = n / 2;
            max_sum = k * (k + 1);
        } else {
            __int128 k = (n + 1) / 2;
            max_sum = (k * k) - 1;
        }

        print128(max_sum * m);
    }
```

```
    return 0;
}
```

Q2: Decrypting the King's Secret Message

A King has encrypted a secret message using a simple substitution-addition cipher. To decrypt it, you must first identify the correct **Key** from a list of potential candidates.

Encryption Rules:

1. Each letter of the alphabet is assigned a numerical value: A=1, B=2, C=3, ..., Z=26.
2. A **Key** is chosen, which is a single uppercase English character. Let the value of this key be K.
3. Every character in the original message is transformed into an integer by adding K to its original alphabet value.
4. The final encrypted message is a sequence of integers separated by spaces.

Decryption Task:
You are given a string containing several potential **candidate keys** and the **encrypted message** (a sequence of integers). You need to find the correct key from the candidate string.

The correct key is the **first character** in the candidate string that satisfies the following condition:

- When its value is subtracted from every integer in the encrypted message, the resulting values must all be valid alphabet positions (integers between **1 and 26** inclusive).

Once the correct key is found, use it to decrypt the integers back into their corresponding uppercase letters and output the original message.

Input Format

- **First Line:** A string representing a list of potential candidate keys (e.g., `"AYBTHC"`).
- **Second Line:** A sequence of space-separated integers representing the encrypted message (e.g., `28 25 31 31 35`).

Output Format

- A single string representing the decrypted original message.

Example Test Cases

Example 1 (From Video)

Input:

```
AYBTHC
28 25 31 31 35
```

Explanation:

1. **Candidate 'A' (Value 1):** 28 - 1 = 27. Invalid (greater than 26).
2. **Candidate 'Y' (Value 25):** 25 - 25 = 0. Invalid (less than 1).
3. **Candidate 'B' (Value 2):** 28 - 2 = 26 (Z), but 35 - 2 = 33. Invalid.
4. **Candidate 'T' (Value 20):**
   - 28 - 20 = 8 → 'H'
   - 25 - 20 = 5 → 'E'
   - 31 - 20 = 11 → 'K'
   - 31 - 20 = 11 → 'K'
   - 35 - 20 = 15 → 'O'
   - All results (8, 5, 11, 11, 15) are within the valid range [1, 26].
     Output:

```
HEKKO
```

Example 2

Input:

```
XYZAB
10 11 12
```

Explanation:

1. **Candidate 'X' (24), 'Y' (25), 'Z' (26):** Subtraction results in negative numbers. Invalid.
2. **Candidate 'A' (Value 1):**
   - 10 - 1 = 9 → 'I'
   - 11 - 1 = 10 → 'J'
   - 12 - 1 = 11 → 'K'
   - All results are valid.
     Output:

```
IJK
```

Constraints

- The candidate key string will only contain uppercase English letters.
- The encrypted message will consist of positive integers.
- There is guaranteed to be at least one valid key in the candidate string.
- $1 \leq$ Length of Message $\leq 10^5$
- $1 \leq$ Number of Candidates ≤ 26

This problem requires identifying a specific shift value (the Key) from a set of candidates that maps a sequence of integers back to the valid range of the English alphabet (1 to 26).

Detailed Code Logic Explanation

1. **Input Parsing**: We receive a string of potential keys and a list of integers. The integers represent $AlphabetValue + KeyValue$.
2. **Constraint Calculation**: To avoid checking every integer for every key, we determine the valid mathematical range for the key K:
   - **Maximum Constraint**: The largest integer in the message ($maxE$) must not exceed $26 + K$. Therefore, $K \geq maxE - 26$.
   - **Minimum Constraint**: The smallest integer in the message ($minE$) must be at least $1 + K$. Therefore, $K \leq minE - 1$.
3. **Key Selection**: We iterate through the candidate string. For each character, we calculate its value ($A = 1, B = 2 \ldots$). The first character whose value falls between the calculated $[lowerBound, upperBound]$ is our decryption key.
4. **Transformation**: For each integer in the encrypted list, we subtract the key value. We then convert this result (1-26) back to a character ($1 \rightarrow' A', 2 \rightarrow' B' \ldots$) using ASCII offsets.

Concept Mapping

- **Linear Search**: Iterating through candidates to find the first match.
- **Constraint Satisfaction**: Using the properties of the alphabet (fixed size 26) to narrow down the search space.
- **Character Arithmetic**: Mapping characters to integers and vice versa using ASCII values (`'A'` is 65 in decimal).
- **Stream/String Parsing**: Converting space-separated input strings into numeric arrays or lists.

C++ Solution

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>
```

```
using namespace std;

int main() {
    string candidates;
    if (!(cin >> candidates)) return 0;

    string line;
    getline(cin >> ws, line);
    stringstream ss(line);

    vector<int> encryptedMsg;
    int num;
    int minEnc = 2e9, maxEnc = -2e9;

    while (ss >> num) {
        encryptedMsg.push_back(num);
        if (num < minEnc) minEnc = num;
        if (num > maxEnc) maxEnc = num;
    }

    if (encryptedMsg.empty()) return 0;

    int lowerBound = maxEnc - 26;
    int upperBound = minEnc - 1;

    int validKeyVal = 0;
    for (char ch : candidates) {
        int kVal = ch - 'A' + 1;
        if (kVal >= lowerBound && kVal <= upperBound) {
            validKeyVal = kVal;
            break;
        }
    }

    string decryptedMsg = "";
```

```
    for (int val : encryptedMsg) {
        int originalVal = val - validKeyVal;
        decryptedMsg += (char)(originalVal + 'A' - 1);
    }

    cout << decryptedMsg << endl;

    return 0;
}
```

12th November Shift 2

Q1: The Mirror Chronicle

Problem Description:
In the subterranean chambers of the Imperial Memory Vault, historians preserve a legendary manuscript known as the **Mirror Chronicle**. The Chronicle is written as a single uninterrupted string of lowercase characters.

Ancient scholars believed that hidden inside every Chronicle lies a **Mirror Fragment** — a contiguous segment of text that reads identically forward and backward (a palindrome). Such fragments were considered sacred because they symbolized temporal symmetry.

The Emperor has issued a decree:
From the given Chronicle, identify the **longest contiguous Mirror Fragment**. If multiple fragments share the same maximum length, return the one that **appears first** in the Chronicle.

Input:

- A single line containing a string of lowercase letters representing the Chronicle.

Output:

- The longest palindromic substring. If multiple palindromes have the same maximum length, output the one that occurs first.

Example Test Case 1

Input:

```
babad
```

Output:

```
bab
```

(Note: "aba" is also a palindrome of the same length, but "bab" appears first in the string.)

Example Test Case 2

Input:

```
forgeeksskeegfor
```

Output:

```
geeksskeeg
```

Solution Logic: Expand Around Center

The objective is to find the longest palindromic substring. A palindrome mirrors itself around its center. For a string of length N, there are $2N - 1$ possible centers for a palindrome:

1. **Odd Centers (N centers):** Each character itself (e.g., in "aba", 'b' is the center).
2. **Even Centers ($N - 1$ centers):** The space between two characters (e.g., in "abba", the space between the two 'b's is the center).

Algorithm Steps:

1. Iterate through each character of the string as a potential center.

2. For each center, attempt to expand outwards as long as the characters on the left and right match.
3. Check both odd-length expansions (center is i) and even-length expansions (center is between i and $i+1$).
4. Maintain two variables: `start` and `maxLen`. Update them **only if** the newly found palindrome length is **strictly greater** than the current `maxLen`. This ensures that if multiple palindromes have the same length, the one appearing first is preserved.
5. **Time Complexity:** $O(n^2)$, where n is the length of the string.
6. **Space Complexity:** $O(1)$, as we only store a few integer variables.

Concept Mapping

- **String Manipulation:** Accessing characters and extracting substrings.
- **Two-Pointer Technique:** Using two indices (`left` and `right`) to expand outwards from a center point.
- **Greedy Approach:** Iterating through all possible centers to ensure the global maximum is found.

C++ Solution

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    string s;
    if (!(cin >> s)) return 0;
    int n = s.length();
    if (n < 2) {
        cout << s << endl;
```

```
        return 0;
    }

    int start = 0, maxLen = 1;

    for (int i = 0; i < n; i++) {
        int l = i, r = i;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                maxLen = r - l + 1;
                start = l;
            }
            l--; r++;
        }

        l = i, r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                maxLen = r - l + 1;
                start = l;
            }
            l--; r++;
        }
    }

    cout << s.substr(start, maxLen) << endl;

    return 0;
}
```

Q2: Maximum GCD Pair with Maximum Absolute Difference

Problem Description:

You are given two integer arrays, **A** and **B**, both of size **N**. Your task is to select one element *a* from array **A** and one element *b* from array **B** such that their Greatest Common Divisor, gcd(*a*, *b*), is as large as possible.

If there are multiple pairs (*a*, *b*) that result in the same maximum GCD value, you must choose the pair that yields the largest absolute difference $|a - b|$.

Output two values: the maximum GCD found and the maximum absolute difference associated with that specific GCD value.

Input Format

1. An integer **N**, representing the number of elements in each array.
2. **N** space-separated integers representing the elements of array **A**.
3. **N** space-separated integers representing the elements of array **B**.

Output Format

- Two space-separated integers: the **Maximum GCD** and the **Maximum Absolute Difference** for that GCD.

Example Test Cases

Test Case 1

Input:

```
3
12 18 6
18 30 10
```

Output:

```
18 0
```

Explanation:

- Possible pairs and their GCDs: (12,18)=6, (12,30)=6, (12,10)=2, (18,18)=18, (18,30)=6, (18,10)=2, (6,18)=6, (6,30)=6, (6,10)=2.

- The maximum GCD is **18**, which comes from the pair (18, 18).
- The absolute difference is |18 - 18| = 0.

Test Case 2

Input:

```
2
12 24
18 36
```

Output:

```
12 24
```

Explanation:

- Pair (12, 18) → GCD: 6, Diff: 6
- Pair (12, 36) → GCD: 12, Diff: 24
- Pair (24, 18) → GCD: 6, Diff: 6
- Pair (24, 36) → GCD: 12, Diff: 12
- The maximum GCD is **12**.
- Two pairs yield GCD 12: (12, 36) with difference 24, and (24, 36) with difference 12.
- The maximum difference for GCD 12 is **24**.

C++ Solution

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    if (!(cin >> n)) return 0;
    vector<int> a(n), b(n);
    int maxVal = 0;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        if (a[i] > maxVal) maxVal = a[i];
    }
    for (int i = 0; i < n; i++) {
        cin >> b[i];
        if (b[i] > maxVal) maxVal = b[i];
    }
    vector<bool> existsA(maxVal + 1, false), existsB(maxVal +
1, false);
    for (int x : a) existsA[x] = true;
    for (int x : b) existsB[x] = true;
    for (int g = maxVal; g >= 1; g--) {
        int minA = 2e9, maxA = -1;
        int minB = 2e9, maxB = -1;
        bool foundA = false, foundB = false;
        for (int m = g; m <= maxVal; m += g) {
            if (existsA[m]) {
                foundA = true;
                if (m < minA) minA = m;
                if (m > maxA) maxA = m;
            }
            if (existsB[m]) {
                foundB = true;
                if (m < minB) minB = m;
                if (m > maxB) maxB = m;
            }
        }
```

```
        if (foundA && foundB) {
            int diff = max(abs(maxA - minB), abs(maxB - min
A));
            cout << g << " " << diff << endl;
            break;
        }
    }
    return 0;
}
```

Solution Logic Explanation

1. **Optimization Strategy:** A brute-force O(N^2) approach (checking every pair) is too slow for large inputs. Instead, we use a **Value-Based approach**. Since the maximum value (M) in the arrays is usually bounded (e.g., 10^5 or 10^6), we iterate through possible GCD values.
2. **The Sieve-like Iteration:**
   - We iterate from the largest possible value `maxVal` down to 1. This value g represents a potential GCD.
   - For a value g to be the GCD of a pair (a, b), both a and b must be multiples of g.
   - We check all multiples of g (g, 2g, 3g, ...) and see if any exist in array A and array B.
3. **Maximizing Difference:**
   - If we find multiples of g in both arrays, then g is a common divisor. Since we are iterating from largest to smallest, the first g we find is the **Maximum GCD**.
   - To find the **Maximum Absolute Difference** for this specific g, we track the smallest and largest multiples of g found in each array (`minA`, `maxA`, `minB`, `maxB`).
   - The maximum difference is either $|maxA - minB|$ or $|maxB - minA|$.

4. **Time Complexity:** The complexity is roughly O(M ln M + N), where M is the maximum value in the input. This is significantly faster than $O(N^2)$ when N is large.

Concept Mapping

Concept	Application in Problem
Divisibility & GCD	Using the property that if g = gcd(a, b), then a and b are multiples of g.
Frequency/Existence Array	Using boolean arrays `existsA` and `existsB` to perform O(1) lookups for elements.
Greedy/Optimal Search	Iterating downwards from the maximum possible value to find the global maximum GCD immediately.
Harmonic Series Complexity	The nested loop (multiples of g) follows the harmonic series ∑(M/g), resulting in O(M log M) efficiency.
Range Extremes	Tracking `min` and `max` within a subset of multiples to calculate absolute differences efficiently.

13th November Shift 1

Q1: Minimum N-Digit Perfect Square Sum Number

Problem Description:

Given an integer N, which signifies the number of digits, your task is to find the **minimum possible N-digit number** such that:

1. The sum of the squares of its digits is a perfect square.
2. The digit '0' **must not** be present anywhere in the number.

A perfect square is an integer that is the square of an integer (e.g., 1, 4, 9, 16, 25, ...).

Input Format

- A single integer N representing the number of digits.

Output Format

- A string representing the smallest N-digit number satisfying the given conditions.

Constraints

- 1 ≤ N ≤ 10^6
- Time Limit: 1.0 second

Example Test Cases

Input (N)	Output (Smallest Number)	Explanation
1	`1`	1^2 = 1 (Perfect Square). Digit '0' is not present.
2	`34`	3^2 + 4^2 = 9 + 16 = 25 (5^2). Smallest 2-digit number without '0's.
3	`122`	1^2 + 2^2 + 2^2 = 1 + 4 + 4 = 9 (3^2). Smallest 3-digit number without '0's.
4	`1111`	1^2 + 1^2 + 1^2 + 1^2 = 4 (2^2). Smallest 4-digit number without '0's.

To find the minimum N-digit number whose sum of squares of digits is a perfect square (without using 0), we should place the smallest digit, 1, in the most significant positions (on the left).

Solution Logic

1. **Mathematical formulation**:
   Let the N-digit number have $N - m$ digits equal to 1, and m digits $d_1, d_2, \ldots, d_m$ where each d_i is in {2, ..., 9}.
   The sum of squares of digits is:

$$S = (N - m) \cdot 1^2 + \sum_{i=1}^{m} d_i^2 = K^2$$

This simplifies to:

$$\sum_{i=1}^{m}(d_i^2 - 1) = K^2 - N$$

Let $R = K^2 - N$ be the remainder to be formed using the increments $(d_i^2 - 1)$. The possible increment values are:

$$\{2^2 - 1, 3^2 - 1, \ldots, 9^2 - 1\} = \{3, 8, 15, 24, 35, 48, 63, 80\}$$

1. **Optimization strategy**:
   - To make the number as small as possible, we must **minimize** m (the number of digits that are not 1). This keeps as many 1s as possible at the front.
   - If two configurations have the same m, choose the one where the m digits (sorted in ascending order) form the smaller lexicographical string.
   - Since N can be up to 10^6, iterate over perfect squares $K^2 \geq N$ and use **dynamic programming** to find the minimum number of increments needed to reach R.
2. **Dynamic Programming**:
   We solve a variation of the Change-Making problem where the "coins" are the possible increments {3, 8, ..., 80}. $dp[v]$ stores the minimum number of digits needed to get a sum of v.

Concept Mapping

Concept	Application
Dynamic Programming	Used to precompute the minimum digits *(m)* and the specific digits required to fill the remainder $K^2 - N$.
Greedy Strategy	Minimizing *m* and sorting the resulting non-'1' digits to ensure the smallest possible number is formed.
Number Theory	Iterating through perfect squares to find a target sum *S* that allows for a valid digit combination.

C++ Solution

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <string>

using namespace std;

int main() {
    long long n;
    if (!(cin >> n)) return 0;

    vector<int> incs = {3, 8, 15, 24, 35, 48, 63, 80};
    vector<int> d_val = {2, 3, 4, 5, 6, 7, 8, 9};
    int max_r = 10000;
    vector<int> dp(max_r + 1, 1e9);
    vector<int> parent(max_r + 1, -1);
    dp[0] = 0;

    for (int i = 0; i < 8; ++i) {
        for (int j = incs[i]; j <= max_r; ++j) {
            if (dp[j - incs[i]] + 1 < dp[j]) {
                dp[j] = dp[j - incs[i]] + 1;
                parent[j] = incs[i];
            }
        }
    }

    int best_m = 1e9;
    vector<int> best_digits;
    long long k = sqrt(n);
    if (k * k < n) k++;

    for (long long i = k; i < k + 500; ++i) {
        long long r = i * i - n;
```

```
        if (r <= max_r && dp[r] <= n) {
            int m = dp[r];
            vector<int> current_digits;
            int temp_r = r;
            while (temp_r > 0) {
                current_digits.push_back(sqrt(parent[temp_r]
+ 1));
                temp_r -= parent[temp_r];
            }
            sort(current_digits.begin(), current_digits.end
());
            best_m = m;
            best_digits = current_digits;
            break;
        }
    }

    string res(n - best_m, '1');
    for (int d : best_digits) res += to_string(d);
    cout << res << endl;

    return 0;
}
```

Q2: Minimum Swaps to Center

Description:

You are given an N × N matrix of integers, where N is always an **odd** integer. Your task is to find the maximum element in the matrix and move it to the center of the matrix.

The "center" of an N × N matrix is defined as the cell at row ⌊N/2⌋ and column ⌊N/2⌋ (using 0-indexing).

To move the element, you can only perform **swaps between adjacent rows or adjacent columns**. Each such swap counts as one move. Find the minimum number of swaps required to place the maximum element at the center.

Note: If there are multiple occurrences of the maximum value, choose the one that requires the minimum number of swaps to reach the center.

Input Format

1. The first line contains an integer N, representing the number of rows and columns.
2. The next N lines each contain N space-separated integers, representing the elements of the matrix.

Output Format

An integer representing the minimum number of swaps required.

Constraints

- N is an odd integer (1 ≤ N ≤ 1001).
- Matrix elements are integers in the range [-10^9, 10^9].

Example Test Cases

Example 1

Input:

```
3
1 2 3
4 9 5
6 7 8
```

Output:

```
0
```

Explanation: The maximum element is `9`, located at index `(1, 1)`. The center of a 3 × 3 matrix is also `(1, 1)`. Since it is already at the center, 0 swaps are needed.

Example 2

Input:

```
5
0 0 0 0 0
0 0 0 0 0
0 25 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Output:

```
1
```

Explanation: The maximum element is `25`, located at index `(2, 1)`. The center of a 5 × 5 matrix is `(2, 2)`.
Minimum swaps = |2 - 2| + |1 - 2| = 0 + 1 = 1.

Example 3

Input:

```
3
10 1 2
3 4 5
6 7 10
```

Output:

```
2
```

Explanation: The maximum value is `10`. It appears at `(0, 0)` and `(2, 2)`.
The center is `(1, 1)`.

- Distance from `(0, 0)` to `(1, 1)`: $|0-1| + |0-1| = 2$ swaps.
- Distance from `(2, 2)` to `(1, 1)`: $|2-1| + |2-1| = 2$ swaps.
  The minimum distance is `2`.

Solution Logic & Concept Mapping

1. **Matrix Representation & Traversal:** The problem involves a 2D grid of size N × N. We use nested loops to iterate through every cell (i, j) where i is the row index and j is the column index.
2. **Identifying the Goal:** In a 0-indexed N × N matrix where N is odd, the center coordinates are always (center, center) where center = $\lfloor N/2 \rfloor$.
3. **The Manhattan Distance Principle:** Swapping adjacent rows moves an element vertically, and swapping adjacent columns moves it horizontally. The minimum number of steps to move an element from (r_1, c_1) to (r_2, c_2) using only these adjacent moves is the **Manhattan Distance**:
   Total Swaps = $|r_1 - r_2| + |c_1 - c_2|$
4. **Handling Multiple Maximums:** We maintain a variable for the current maximum value found and the minimum distance seen so far.
   - If a new value is strictly greater than the current `max`, we update the `max` and reset the `minDistance`.
   - If a value is equal to the current `max`, we calculate its distance to the center and update `minDistance` only if it is smaller than the previous one.

C++ Solution

```
#include <iostream>
#include <vector>
#include <cmath>
#include <climits>

using namespace std;

int main() {
```

```
    int n;
    if (!(cin >> n)) return 0;

    int center = n / 2;
    long long maxVal = LLONG_MIN;
    int minSwaps = INT_MAX;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            long long currentVal;
            cin >> currentVal;

            int dist = abs(i - center) + abs(j - center);

            if (currentVal > maxVal) {
                maxVal = currentVal;
                minSwaps = dist;
            } else if (currentVal == maxVal) {
                if (dist < minSwaps) {
                    minSwaps = dist;
                }
            }
        }
    }

    cout << minSwaps << endl;

    return 0;
}
```

13th November Shift 2

Q1: Kitchen Task Scheduler

Problem Description:

You are a chef in a high-paced kitchen. You are given a list of tasks to complete, where each task is represented by a character (e.g., 'A', 'B', 'C'). Each task represents the cooking of a specific dish.

Every dish takes exactly **1 unit of time** to cook. However, there is a constraint: after cooking a specific dish, you must wait for a **cooldown period of N time units** before you can cook the same dish again.

During this cooldown period, you can cook other available dishes from your list. If no other dish is available to cook during a particular time unit (because all remaining tasks are either completed or currently in their cooldown period), you must remain idle for that unit of time.

Your goal is to complete all the given tasks in the **minimum amount of time possible**.

Requirement

Return the **sequence** of dishes cooked, including the string **"NOTHING"** for every time unit spent idling.

Input Format

1. An array of characters `tasks` representing the dishes to be cooked.
2. An integer `N` representing the cooldown period.

Output Format

- A list of strings representing the sequence of execution.

Example Test Cases

Example 1:

- **Input:** `tasks = ['A', 'A', 'A', 'B', 'B', 'B']`, `N = 2`
- **Output:** `['A', 'B', 'NOTHING', 'A', 'B', 'NOTHING', 'A', 'B']`
- **Explanation:**
  - **Time 0:** Cook 'A'. 'A' goes into a 2-unit cooldown (Time 1 and 2).

- **Time 1:** Cook 'B'. 'B' goes into a 2-unit cooldown (Time 2 and 3).
- **Time 2:** Both 'A' and 'B' are in cooldown. No other dishes are left. Output "NOTHING".
- **Time 3:** 'A' is ready. Cook 'A'. New cooldown for 'A' (Time 4 and 5).
- **Time 4:** 'B' is ready. Cook 'B'. New cooldown for 'B' (Time 5 and 6).
- **Time 5:** Both 'A' and 'B' are in cooldown. Output "NOTHING".
- **Time 6:** 'A' is ready. Cook 'A'.
- **Time 7:** 'B' is ready. Cook 'B'.
- All tasks are completed in 8 units of time.

Example 2:

- **Input:** `tasks = ['A', 'A', 'A', 'B', 'B', 'B']`, `N = 0`
- **Output:** `['A', 'A', 'A', 'B', 'B', 'B']`
- **Explanation:** Since the cooldown is 0, you can cook all dishes consecutively without any idle time.

Example 3:

- **Input:** `tasks = ['A', 'B', 'C', 'A', 'B', 'C']`, `N = 3`
- **Output:** `['A', 'B', 'C', 'NOTHING', 'A', 'B', 'C']`
- **Explanation:** After cooking A, B, and C, at time unit 3, 'A' is still in its 3-unit cooldown (it will be ready at time unit 4). Since no other tasks are ready, you stay idle ("NOTHING") for one unit.

Solution Logic & Concept Mapping

1. Concept Mapping:

- **Frequency Tracking:** Using a basic Map or Dictionary to store how many of each dish still need to be cooked.
- **Cooldown Tracking:** Using a Map or Dictionary to store the last time index a specific dish was cooked.

- **Greedy Selection:** At every time unit, we scan all possible dishes. Among those whose cooldown has expired, we pick the one that has the highest remaining count. This ensures we finish the most "difficult" tasks first to minimize idle time.
- **Simulation:** Stepping through time unit by unit (0, 1, 2...) until the total number of tasks remaining is zero.

2. Logic Explanation:

1. **Count Initial Frequencies:** Count how many times each dish (character) appears.
2. **Initialize Last Cooked Time:** Set the "last cooked" time for all unique dishes to a very small number (like $-N-1$) so they are all available at Time 0.
3. **Step-by-Step Simulation:**
   - In each time unit, look for an available dish. A dish is available if its `count > 0` and `(CurrentTime - LastTimeCooked) > N`.
   - If multiple dishes are available, pick the one with the highest `count`.
   - If a dish is picked: decrement its `count`, update its `LastTimeCooked` to the `CurrentTime`, and add the dish name to the result list.
   - If no dish is available: add **"NOTHING"** to the result list.
4. **Completion:** Repeat until all dishes are cooked.

C++ Solution

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <sstream>

using namespace std;

int main() {
```

```
    string line, task;
    getline(cin, line);
    stringstream ss(line);
    vector<string> tasksArray;
    while (ss >> task) tasksArray.push_back(task);

    int n;
    cin >> n;

    map<string, int> counts;
    map<string, int> lastCooked;
    for (string s : tasksArray) {
        counts[s]++;
        lastCooked[s] = -n - 1;
    }

    vector<string> result;
    int currentTime = 0;
    int totalRemaining = tasksArray.size();

    while (totalRemaining > 0) {
        string bestTask = "";
        int maxFreq = -1;

        for (auto const& [name, freq] : counts) {
            if (freq > 0 && (currentTime - lastCooked[name])
> n) {
                if (freq > maxFreq) {
                    maxFreq = freq;
                    bestTask = name;
                }
            }
        }

        if (bestTask != "") {
            result.push_back(bestTask);
```

```
            counts[bestTask]--;
            lastCooked[bestTask] = currentTime;
            totalRemaining--;
        } else {
            result.push_back("NOTHING");
        }
        currentTime++;
    }

    cout << "[";
    for (int i = 0; i < result.size(); i++) {
        cout << "'" << result[i] << "'" << (i == result.size
() - 1 ? "" : ", ");
    }
    cout << "]" << endl;

    return 0;
}
```

Q2: Rounding off Decimals

Problem Description:

You are a shopkeeper and have to round up the cost of the item to the nearest two decimal which is being purchased from your store.

For eg.

if a person's total is 9.7854 then the round up value will be 9.79

And similarly for 3.1412 will be 3.14

Solution Logic & Concept Mapping

1. **Solution Logic:**
    - **Input Handling:** Read the floating-point number from the standard input.

- **Rounding Mechanics:** The goal is to round to 2 decimal places using "Round Half Up" (where 0.5 rounds to 1).
- **Precision Management:** Since computers represent floating-point numbers in binary, simple math (like `100`) can sometimes lead to tiny precision errors (e.g., `0.005` might be stored as `0.00499999`).
- **Formatting:** The final step involves ensuring the output shows exactly two decimal places (e.g., `5.1` becomes `5.10`) using string formatting tools provided by the language.

2. **Concept Mapping:**
   - **Data Types:** Floating-point precision (`double`, `float`, `Decimal`).
   - **Rounding Modes:** Standard Mathematical Rounding vs. Banker's Rounding.
   - **String Formatting:** Using format specifiers like `%.2f` to control output display.
   - **Library Usage:** Utilizing `iomanip` (C++), `Decimal` (Python), and `printf` (Java).

C++ Solution

```
#include <iostream>
#include <iomanip>
#include <cmath>

using namespace std;

int main() {
    long double n;
    if (cin >> n) {
        cout << fixed << setprecision(2) << n << endl;
    }
    return 0;
}
```

14th November

Q1: Sum of Even Numbers up to N

Problem Description:
Given a positive integer `N`, write a program to find the sum of all even numbers from 1 to `N` (inclusive).

Input Format:

- A single integer `N` representing the upper limit of the range.

Output Format:

- A single integer representing the sum of all even integers in the range `[1, N]`.

Constraints:

- 1 ≤ N ≤ 10^9 (Note: Using a mathematical formula is more efficient for large values of N).

Sample Test Cases:

Example 1:

Input:

```
6
```

Output:

```
12
```

Explanation:
The even numbers from 1 to 6 are 2, 4, and 6.
Sum = 2 + 4 + 6 = 12.

Example 2:

Input:

```
5
```

Output:

```
6
```

Explanation:
The even numbers from 1 to 5 are 2 and 4.
Sum = 2 + 4 = 6.

Here are the working code solutions for the problem **"Sum of Even Numbers up to N"** in Java, Python, and C++, followed by the logic explanation and concept mapping.

C++ Solution

```
#include <iostream>

using namespace std;

int main() {
    long long n;
    if (cin >> n) {
        long long k = n / 2;
        long long result = k * (k + 1);
        cout << result << endl;
    }
    return 0;
}
```

Solution Logic Explanation

The goal is to find the sum of all even numbers in the range [1, N]. The sequence of even numbers is an **Arithmetic Progression (AP)**: 2, 4, 6, 8, ..., 2k, where 2k is

the largest even number less than or equal to N.

1. **Identify the number of terms (k):**
   Since every second number is even, the total count of even numbers up to N is simply the integer division of N by 2.
   $k = \lfloor N / 2 \rfloor$
2. **Derive the Formula:**
   The sum of the first k even numbers is:
   $S = 2 + 4 + 6 + \ldots + 2k$
   Factor out 2:
   $S = 2 \times (1 + 2 + 3 + \ldots + k)$
   The sum of the first k natural numbers is given by the formula $k(k+1)/2$.
   Substituting this in:
   $S = 2 \times k(k+1)/2 = k(k+1)$
3. **Efficiency:**
   Instead of using a loop which would take O(N) time, this mathematical approach calculates the result in **O(1) (constant time)**.
4. **Handling Large Numbers:**
   For N up to 10^9, the result can exceed the capacity of a standard 32-bit integer (which maxes out at $\approx 2 \times 10^9$). Therefore, 64-bit integers (`long` in Java, `long long` in C++) are used to prevent overflow.

Concept Mapping

Concept	Application in this Solution
Arithmetic Progression (AP)	Recognizing that even numbers form a sequence with a common difference of 2.
Arithmetic Sum Formula	Using the sum of natural numbers formula to derive $k(k+1)$.
Integer Division	Using `N / 2` to find the total count of even integers.
Data Type Precision	Utilizing 64-bit integers to handle results up to 10^{17} to avoid overflow.
Time Complexity Optimization	Transitioning from an O(N) loop to an O(1) mathematical calculation.

Q2: Median of Two Sorted Arrays

Problem Statement:
You are given two sorted integer arrays, `nums1` and `nums2`, of sizes `n` and `m` respectively. Your task is to find the median of the two sorted arrays after merging them.

Constraints:

- `nums1` and `nums2` are sorted in non-decreasing order.
- The sizes `n` and `m` can be different.
- The arrays may contain duplicate elements.

Example Test Cases

Example 1

Input:

```
nums1 = [1, 3, 5]
nums2 = [2, 4, 6, 7, 8]
```

Output:

```
4.5
```

Explanation:
The merged sorted array is `[1, 2, 3, 4, 5, 6, 7, 8]`.
The total number of elements is 8 (even).
The two middle elements are at the 4th and 5th positions: `4` and `5`.
Median = (4 + 5) / 2 = 4.5.

Example 2

Input:

```
nums1 = [1, 3, 5]
nums2 = [2, 4, 6, 7]
```

Output:

```
4.0
```

Explanation:
The merged sorted array is `[1, 2, 3, 4, 5, 6, 7]`.
The total number of elements is 7 (odd).
The middle element is at the 4th position: `4`.
Median = `4.0`.

Example 3

Input:

```
nums1 = [1, 2]
nums2 = [3, 4]
```

Output:

```
2.5
```

Explanation:
The merged sorted array is `[1, 2, 3, 4]`.
Total elements = 4 (even).
Middle elements are `2` and `3`.
Median = (2 + 3) / 2 = 2.5.

C++ Solution

```
#include <iostream>
#include <vector>
#include <iomanip>
```

```
using namespace std;

int main() {
    int n, m;

    if (!(cin >> n)) return 0;
    vector<int> nums1(n);
    for (int i = 0; i < n; i++) cin >> nums1[i];

    if (!(cin >> m)) return 0;
    vector<int> nums2(m);
    for (int i = 0; i < m; i++) cin >> nums2[i];

    int i = 0, j = 0;
    int m1 = -1, m2 = -1;
    int totalLen = n + m;

    for (int count = 0; count <= totalLen / 2; count++) {
        m2 = m1;
        if (i < n && j < m) {
            if (nums1[i] < nums2[j]) {
                m1 = nums1[i++];
            } else {
                m1 = nums2[j++];
            }
        } else if (i < n) {
            m1 = nums1[i++];
        } else {
            m1 = nums2[j++];
        }
    }

    cout << fixed << setprecision(1);
    if (totalLen % 2 == 1) {
        cout << (double)m1 << endl;
```

```
    } else {
        cout << (double)(m1 + m2) / 2.0 << endl;
    }

    return 0;
}
```

Solution Logic Explanation

1. **Input Parsing:** We take the size of the first array, followed by its elements, and then do the same for the second array.
2. **The Two-Pointer Strategy:**
   - Since both arrays are already sorted, we don't need to actually merge them into a new array (which would take O(n+m) space).
   - We use two pointers, `i` and `j`, starting at the beginning of each array.
   - We simulate the merge process by comparing `nums1[i]` and `nums2[j]`, "moving" to the next element of the merged array by incrementing the pointer of the smaller value.
3. **Tracking Middle Elements:**
   - We need to reach the "middle" index, which is `(n+m)/2`.
   - `m1` stores the current element we are looking at in our "virtual" merged array.
   - `m2` stores the previous element.
4. **Final Result:**
   - If the total length is **odd**, the median is simply `m1` (the middle element).
   - If the total length is **even**, the median is the average of `m1` and `m2` (the two middle elements).

Concept Mapping

Concept	Application in this Problem
Two-Pointers	Used to traverse two sorted structures simultaneously without extra space.
Simulated Merge	Emulating the "Merge" step of the Merge Sort algorithm to find specific indices.
Median Calculation	Handling even vs. odd total counts to determine the middle point(s).
Time Complexity (O(n+m))	Traversing only up to the halfway point of the combined elements.
Space Complexity (O(1))	Avoiding the creation of a third array to save memory.

20th November

Q1: Maximize XOR Sum with K-Set Bits

Description:
You are given an array of N integers and an integer K. Your task is to find an integer X such that:

1. X has exactly K set bits (1s) in its binary representation.
2. The total sum S is maximized, where S = sum of (X XOR arr[i]) for i from 0 to N-1.
   (Note: XOR denotes the bitwise XOR operation).

If there are multiple values of X that result in the same maximum sum, you must return the **minimum** possible value of X.

Input Format:

- The first line contains an integer N, representing the size of the array.
- The second line contains N space-separated integers representing the elements of the array.
- The third line contains an integer K, representing the required number of set bits in X.

Output Format:

- A single integer representing the minimum value of X that satisfies the conditions.

Constraints:

- 1 ≤ N ≤ 10^5
- 0 ≤ arr[i] ≤ 10^9
- 1 ≤ K ≤ 30
- X should be treated as a non-negative 31-bit integer (0 ≤ X < 2^31).

Example Test Cases

Example 1

Input:

```
3
1 2 3
1
```

Output:

```
1073741824
```

(Note: In a real exam environment, there is usually a limit on the bits, e.g., 30 bits. If we assume the search space is up to 2^30, the highest bit provides the largest sum increase because count0 > count1 for that bit.)

Example 2 (Simplified Logic)

Input:

```
3
1 2 3
2
```

Explanation:

- Array in binary: `1 (01)`, `2 (10)`, `3 (11)`.
- We need to pick 2 bits to set in X to maximize sum of (X XOR arr[i]).
- The bits that provide the most "gain" are the ones where the array has more 0s than 1s at that position.
- If we consider bits up to the 30th bit: The highest bits (like 2^30, 2^29, etc.) have three 0s and zero 1s in the array. Setting these bits in X gives the maximum sum.
- To get the **minimum X**, we pick the K bits that give the highest gain. If gains are equal, we pick the lower bit indices to keep the value of X small.

This problem is a variation of the **"Maximize XOR Sum"** problem, which is common in competitive programming. The key insight is that the bitwise XOR operation affects each bit position independently.

Solution Logic & Concept Mapping

1. **Bitwise Independence**: The total sum of (X XOR arr[i]) can be broken down into the sum of contributions from each bit position j (from 0 to 30).
2. **Contribution Calculation**:
   - For a bit position j, let count1 be the number of elements in the array with the j-th bit set, and count0 be the number of elements with the j-th bit as 0.
   - If we set the j-th bit of **X** to **1**, it flips the bits in the array. The contribution to the total sum at this bit position becomes count0 × 2^j.
   - If we set the j-th bit of **X** to **0**, the contribution remains count1 × 2^j.
3. **Greedy Strategy (Gain)**:
   - The "Gain" of setting the j-th bit of X to 1 is: (count0 - count1) × 2^j.
   - To maximize the total sum, we want to pick K bit positions that offer the highest gains.
4. **Tie-Breaking for Minimum X**:

- If two bit positions provide the exact same gain, we must pick the **smaller bit index** to ensure the resulting value of X is minimized.
- *Note*: Because of the 2^j factor, gains are rarely equal unless the term (count0 - count1) is zero.

C++ Solution

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

struct BitInfo {
    long long gain;
    int index;
};

bool compareBits(const BitInfo& a, const BitInfo& b) {
    if (a.gain != b.gain) {
        return a.gain > b.gain; // Higher gain first
    }
    return a.index < b.index; // Smaller index first for mini
mum X
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    vector<long long> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
```

```
    int k;
    cin >> k;

    vector<BitInfo> bitGains(31);

    for (int j = 0; j <= 30; j++) {
        long long c1 = 0;
        for (int i = 0; i < n; i++) {
            if ((arr[i] >> j) & 1) c1++;
        }
        long long c0 = n - c1;
        bitGains[j].gain = (c0 - c1) * (1LL << j);
        bitGains[j].index = j;
    }

    sort(bitGains.begin(), bitGains.end(), compareBits);

    long long x = 0;
    for (int i = 0; i < k; i++) {
        x |= (1LL << bitGains[i].index);
    }

    cout << x << endl;

    return 0;
}
```

Concept Mapping

- **Greedy Algorithm**: Picking the bits with the highest contribution to the sum at each step.
- **Bit Manipulation**: Using shifts (`>>`, `<<`) and bitwise OR (`|`) to analyze and construct numbers.

- **Sorting with Custom Comparators**: Necessary to handle the "maximum sum" primary condition and "minimum value" secondary condition.
- **Complexity**:
  - **Time**: O(31 × N) to count bits + O(31 log 31) to sort. This is effectively **O(N)**, which easily passes within the 1-second limit for N = 10^5.
  - **Space**: O(1) additional space beyond the input array, as the bit-tracking structures are constant size (31).

Q2: Optimal Yearly Project Selection

You are managing projects over **L** years. For each year, you are given a maximum capital budget and a minimum required production level. There are **N** available projects you can choose from.

Constraints for each year i:

1. You can select **at most one** project in any given year.
2. A project j can only be selected in year i if:
   - The project's **Capital Cost** is less than or equal to the year's budget (C_i).
   - The project's **Production Value** is greater than or equal to the year's requirement (P_i).
3. Projects can be reused in different years (as seen in the example).

Goal:
Given the Net Present Value (NPV) of each project for each specific year, find the maximum total NPV you can achieve across all L years.

Input Format

1. **L**: Number of years.
2. **N**: Number of projects.
3. **Budgets[]**: An array of size L, where `Budgets[i]` is the max capital for year i.

4. **Requirements[]**: An array of size L, where `Requirements[i]` is the min production for year i.
5. **ProjectCosts[]**: An array of size N, representing the capital cost of each project.
6. **ProjectProductions[]**: An array of size N, representing the production value of each project.
7. **NPV[][]**: A 2D array of size L × N, where `NPV[i][j]` is the profit of project j if performed in year i.

Output Format

- An integer representing the maximum total NPV summed over all L years.

Example Test Case

Input:

```
L = 2, N = 3
Budgets = [20, 15]
Requirements = [25, 30]
ProjectCosts = [25, 21, 14]
ProjectProductions = [20, 26, 31]
NPV Table:
    P1  P2  P3
L1 [10, 15, 12]
L2 [8,  18, 20]
```

Step-by-Step Explanation:

- **Year 1 (Budget ≤ 20, Production ≥ 25):**
  - P1: Cost 25 (>20) — **Invalid**
  - P2: Cost 21 (>20) — **Invalid**
  - P3: Cost 14 (≤ 20) and Production 31 (≥ 25) — **Valid**. NPV = 12.
- **Year 2 (Budget ≤ 15, Production ≥ 30):**

- P1: Cost 25 (>15) — **Invalid**
- P2: Cost 21 (>15) — **Invalid**
- P3: Cost 14 (≤ 15) and Production 31 (≥ 30) — **Valid**. NPV = 20.

Total NPV: 12 + 20 = 32.

Output:

```
32
```

Solution Logic & Concept Mapping

1. Problem Type: Greedy Optimization with Constraint Satisfaction
The problem requires us to make an optimal choice (maximum NPV) for each year independently, subject to specific constraints (Budget and Production). Since the choice made in Year 1 does not restrict the choices available in Year 2, a **Greedy Approach** per year yields the globally optimal solution.

2. Decision Logic:
For every year *i* (from 0 to L-1):

- Identify all projects *j* that satisfy: `ProjectCosts[j] <= Budgets[i]` AND `ProjectProductions[j] >= Requirements[i]`.
- Among these valid projects, select the one with the highest `NPV[i][j]`.
- Add this maximum value to a running total.

3. Complexity Analysis:

- **Time Complexity:** O(L × N), where L is the number of years and N is the number of projects. We iterate through every project for every year.
- **Space Complexity:** O(L × N) to store the NPV matrix, or O(N) if processing input streamingly.

C++ Solution

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>

using namespace std;

int main() {
    int L, N;
    if (!(cin >> L >> N)) return 0;

    vector<int> budgets(L);
    for (int i = 0; i < L; i++) cin >> budgets[i];

    vector<int> requirements(L);
    for (int i = 0; i < L; i++) cin >> requirements[i];

    vector<int> projectCosts(N);
    for (int i = 0; i < N; i++) cin >> projectCosts[i];

    vector<int> projectProductions(N);
    for (int i = 0; i < N; i++) cin >> projectProductions[i];

    vector<vector<int>> npv(L, vector<int>(N));
    for (int i = 0; i < L; i++) {
        for (int j = 0; j < N; j++) {
            cin >> npv[i][j];
        }
    }

    long long totalNPV = 0;

    for (int i = 0; i < L; i++) {
        int maxYearNPV = 0;
        for (int j = 0; j < N; j++) {
            if (projectCosts[j] <= budgets[i] && projectProdu
ctions[j] >= requirements[i]) {
                if (npv[i][j] > maxYearNPV) {
                    maxYearNPV = npv[i][j];
```

```
                }
            }
        }
        totalNPV += maxYearNPV;
    }

    cout << totalNPV << endl;

    return 0;
}
```

Concept Mapping

Feature	Mapping to this Problem
Optimization Goal	Maximizing the sum of Profits (NPV).
Constraints	Upper bound on Capital (C), Lower bound on Production (P).
Data Structure	2D Arrays (Matrix) for NPV values, 1D Arrays for Project/Year attributes.
Algorithm Strategy	**Independent Greedy Choice**: Since picking a project in Year 1 doesn't deplete a shared global resource (the PDF says we just check against each year's limit), we solve each year optimally in isolation.
Edge Case Handling	If no project meets the criteria for a specific year, the `maxYearNPV` remains 0, ensuring the sum remains correct.